

CLABTOOLKIT: Reducing Repetitive Tasks in Neuroimaging Research

26.10.2026

Yasser Alemán-Gómez

Connectomics Lab, Lausanne University Hospital (CHUV)

CLABTOOLKIT Documentation: https://clabtoolkit.readthedocs.io
CLABTOOLKIT Source code: https://github.com/connectomicslab/clabtoolkit

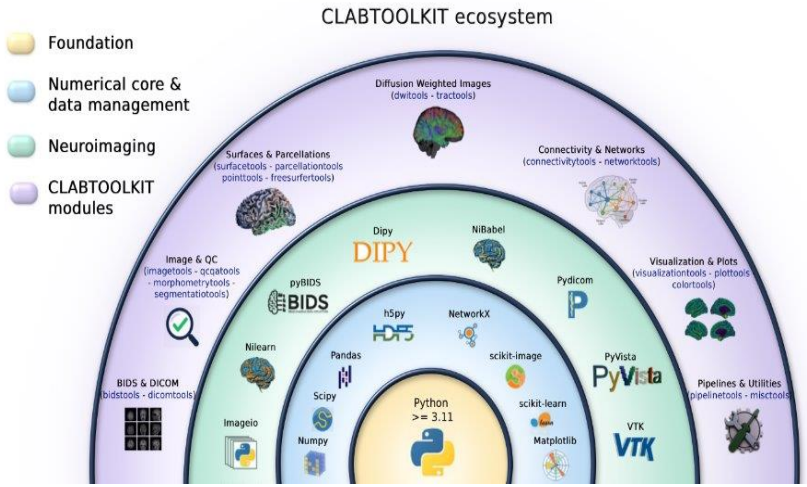
Motivation and objective

- **Motivation:**
  - **Routine** neuroimaging **operations** are **re-implemented** in **every project**. These ad hoc versions are rarely tested, which risks subtle **errors**, hurts **reproducibility** and raises the barrier for new users.

- **AIM:**
  - To **provide** a single, tested **layer** of shared **utilities** in Python that **removes repetitive**, error-prone **tasks**, building more robust, reproducible and FAIR workflows.

CLABTOOLKIT Ecosystem

Dependencies ecosystem



- **Main external and optional dependencies**
  - FreeSurfer
  - ANTs

The toolkit ships 5 JSON files

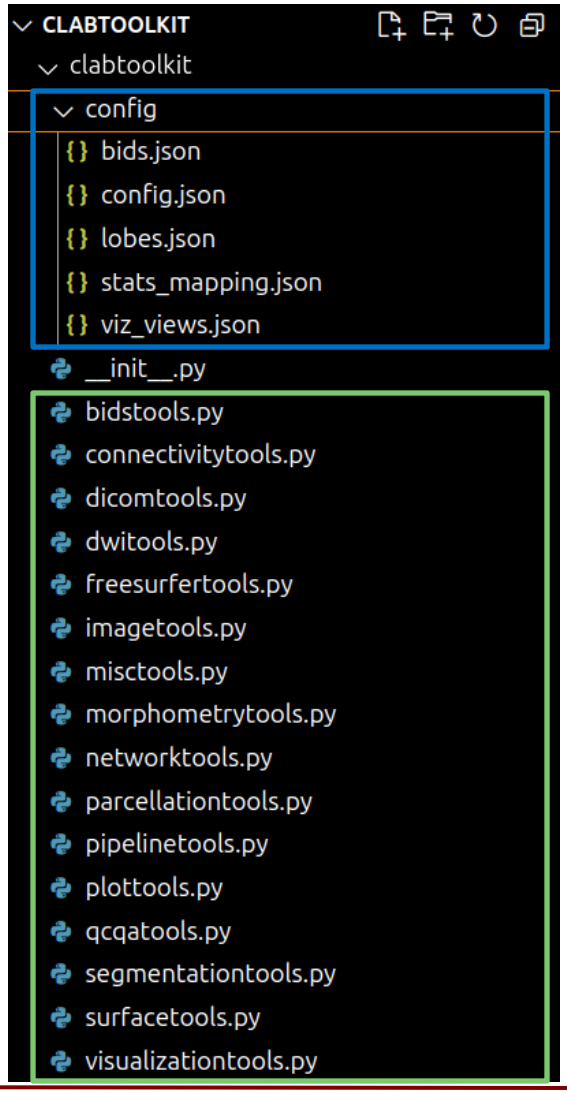
- **config.json:** *Units configuration*
  - Network metrics for binary or weighted graphs
  - Units
- **viz_views.json:** *Camera views, layouts and themes*
  - Windows size
  - Font colors
  - Layouts and themes
  - Views
- **lobes.json:** *Custom lobar grouping*
  - Lobar grouping
- **stats_mapping.json:** *Adapting the FreeSurfer parser*
  - global:
  - aseg:
  - cortex:
- **bids.json:** *Handle BIDS entities*
  - Allowed raw and derivatives entities
  - Correct suffixes

CLABTOOLKIT Package architecture

- **Total Statistics:**
  - **19** modules
  - **27, 681** lines of code
  - **15** classes including **329** methods
  - **358** functions

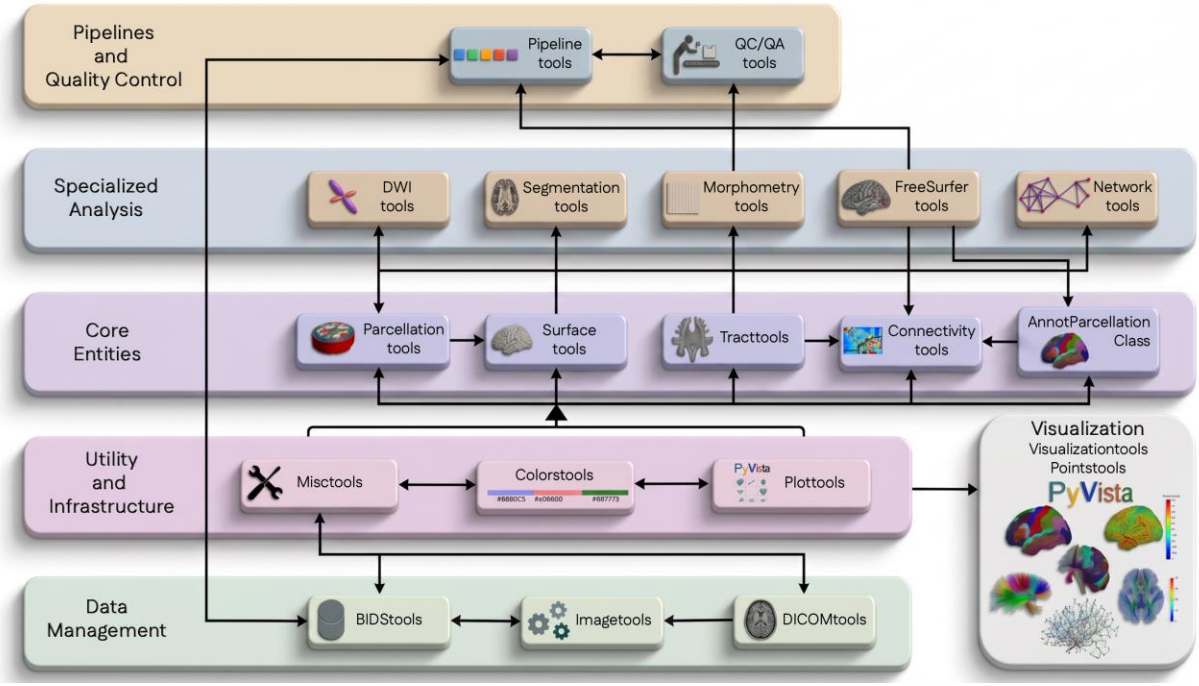
- **Key Modules:**
  - parcellationtools.py - **2828** lines
    (**2** classes, **66** methods)
  - visualizationtools.py - **2,548** lines
    (**1** class, **29** methods, **42** functions)
  - misctools.py - **4,947** lines
    (**1** classes, **1** method, **55** functions)

- **Principal Classes:**
  1. **BrainPlotter** (visualizationtools.py) - **29** methods
  2. **Surface** (surfacetools.py) - **47** methods
  3. **Parcellation** (parcellationtools.py) - **66** methods
  4. **Tractogram** (tracttools.py) - **29** methods



CLABTOOLKIT Package architecture

Architecture diagram of the CLABTOOLKIT modules



- **Architecture patterns:**
  - Specialized layered structure
  - Small set of core entities
  - Shared infrastructure
  - Configuration by JSON files
- **Strengths:**
  - Modular design
  - BIDS compliance
  - Reach visualization capabilities
- **Potencial issues:**
  - Circular dependencies (handled with conditional imports)
  - Large modules

CLABTOOLKIT Demonstration

Table of Contents

Module: ParcellationTools (1 Classes, 30 Methods, 0 Functions)

- 🔷 **1.** `Parcellation`: Class for analyzing and manipulating brain parcellations
  - 🔸 1.1. Go to `keep_by_name`: Filter the parcellation by region names
  - 🔸 1.2. Go to `compute_centroids`: Compute centroids, number of voxels, and volume for each region
  - 🔸 1.3. Go to `get_regionwise_timeseries`: Compute region-wise time series
  - 🔸 1.4. Go to `compute_volume_table`: Calculate volumes for each region
  - 🔸 1.5. Go to `surface_extraction`: Extract 3D surface meshes from parcellation regions

---

Module: SurfaceTools (1 Classes, 50 Methods, 1 Functions)

- 🔷 **2.** `Surface`: Main class for FreeSurfer surface manipulation and visualization. Initialize Surface object with geometry file
  - 🔸 2.1. Go to `load_scalar_map`: Load morphometric data onto surface
  - 🔸 2.2. Go to `load_annotation`: Load FreeSurfer annotation over the surface
  - 🔸 2.3. Go to `merge_surfaces`: Merge this surface with others into a single surface
  - 🔸 2.4. Go to `plot`: Visualize surface with customizable rendering options

---

Module: BIDsTools (34 Functions)

- 🔸 3.1. Go to `generate_bids_tree`: Generate an MS-DOS tree-style visualization of a BIDS folder structure
- 🔸 3.2. Go to `get_bids_database_table`: Generate a comprehensive summary table of all neuroimaging files in a BIDS dataset

---

Module: MiscTools (2 Classes, 6 Methods, 76 Functions)

- 🔸 4.1. Go to `values2colors`: Map numerical values to colors using a specified colormap with optional inversions.
- 🔸 4.2. Go to `build_indices`: Function to build the indexes from a range vector that can contain integers, tuples, lists or strings
- 🔸 4.3. Go to `generate_container_command`: This function generates the command to run a bash command inside a container
- 🔸 4.4. Go to `show_module_contents`: Displays all classes and functions in a given module with colored formatting